ПП ПМ.13 · Раздел 1

1. Полное задание

1

Назначение и результат

Подготовить завершённую игру назначенной предметной области к выпуску. Адаптировать интерфейс, добавить обучение, согласованные переходы и сведения об игре. Сохранить три уровня, собственное правило, победу, поражение, повтор и постоянные данные игрока. Подготовить представление продукта и доказательства проверок.

2

Исходный проект

Использовать собственный готовый Unity-проект с тремя уровнями и правилом из отдельного списка предметных областей. Зафиксировать исходный коммит, текущие рекорды, доступные уровни и настройки. Если проект утрачен, восстановить его из собственного репозитория до начала изменений. Если нужного проекта нет, согласовать исходную игру с преподавателем; контрольный пример не заменяет назначенный вариант.

3

Среда и границы

Использовать Unity версии проекта, редактор C#, Git, Word и браузер. Контрольный пример подготовлен для 6000.6.2f1. Приёмка основного результата выполняется в Game View — окне игры в редакторе. Исполняемая сборка допускается при наличии модуля платформы; Android и публикация в магазине не обязательны. Магазин заменить статичным макетом страницы продукта.

4

Порядок работы

Зафиксировать входную игру → проверить размеры и безопасную область → добавить переходы и обучение → оформить сведения и реальные кадры → подготовить материалы Release → проверить понятность и игровой цикл → передать один собственный проект и заполненный отчёт. Образец «Кристальный маршрут» показывает приёмы; повторять его как вторую обязательную игру не требуется.

ПП ПМ.13 · Раздел 2

1.1. Интерфейс и первый запуск

1

Размеры и безопасная область

Проверить меню, HUD — игровые индикаторы, паузу, результат, настройки, обучение и «Об игре» при 1280×720 (16:9), 1440×720 (18:9) и 1024×768 (4:3). Сохранить снимки каждого режима и таблицу экранов/разрешений. Отдельно включить имитацию вырезов: элементы остаются внутри доступной области. Основная ориентация — альбомная.

2

Три подсказки

При первом запуске показать три короткие карточки: управление; особое правило; ошибка и завершение. Обеспечить переход «Далее», завершение «В меню», пропуск и повтор через «Как играть». Сохранить признак просмотра отдельно от игрового прогресса. Текст и индикаторы должны описывать именно правило назначенного варианта.

3

Переходы и сведения

Добавить плавное появление панелей, работающее при паузе. До окончания появления отключить кнопки панели, скрытые панели исключить из ввода. В «Об игре» показать название, версию, фактическое авторство и происхождение ресурсов; согласовать версию с Player Settings и README.

4

Понятность

Дать человеку, впервые видящему игру, начать попытку без подсказок, выполнить правило, поставить паузу и продолжить. Записать наблюдаемое затруднение, исправление и повторный результат. Если участника нет, выполнить самостоятельный проход и честно отметить ограничение; вымышленные наблюдения не допускаются.

ПП ПМ.13 · Раздел 3

1.2. Представление продукта

1

Графика и игровые кадры

Подготовить icon.png 512×512 и promo.png 1280×720. Сохранить не менее шести реальных игровых кадров: меню, обучение, процесс, пауза, результат и «Об игре». Это учебные размеры, а не правила конкретного магазина. Иконка и рекламная композиция могут быть самостоятельной графикой; игровые снимки должны показывать фактическую игру.

2

Тексты

Создать краткое описание до 80 знаков и полное описание 600–1200 знаков, инструкцию игроку, сведения об авторах и ресурсах, список изменений и известные ограничения. Обещания сверить с действующей игрой. Данные о выпуске продукта относятся к заданию; их не смешивать со служебным отчётом проверки методички.

3

Макет страницы

Подготовить статичный макет с названием, иконкой, кратким и полным описанием, реальными снимками, версией и авторами. Достаточно локальной HTML-страницы, открываемой в браузере; размещать игру в магазине или пользоваться платным сервисом не требуется.

4

Демонстрация

Подготовить сценарий длительностью 60–90 секунд: первое знакомство, особое правило, победа, повтор, «Об игре». Приложить 6–8 статичных кадров с подписями и интервалами показа. Видеоролик можно приложить при наличии средства записи; раскадровку не называть видеозаписью.

ПП ПМ.13 · Раздел 4

1.3. Приёмочные проверки

1

Экраны · T01–T04

T01: меню, HUD, пауза, результат, настройки, обучение и «Об игре» видимы в 16:9, 18:9, 4:3. T02: включённая имитация отступов оставляет кнопки внутри безопасной области. T03: кнопки доступны мышью. T04: скрытая панель не перехватывает нажатия.

2

Обучение · T05–T08

T05: новый ключ обучения показывает первую подсказку. T06: «Далее» проходит три шага, последний возвращает меню. T07: пропуск и завершение не сбрасывают рекорды; после нового Play автоматического показа нет. T08: «Как играть» повторно открывает шаг 1.

3

Полировка · T09–T12

T09: панель паузы появляется при Time.timeScale = 0. T10: до конца появления нет повторных кликов. T11: версия на экране совпадает с Player Settings и README. T12: каждый PNG открывается и показывает фактическое состояние игры.

4

Выпуск · T13–T16

T13: наблюдение за пользователем и повторная проверка описаны честно. T14: правило своего варианта понятно без устного объяснения. T15: сохранены три уровня, победа, поражение и прогресс. T16: новая копия репозитория запускается; комплект относится к одному коммиту.

ПП ПМ.13 · Раздел 5

1.4. Состав сдачи

1

Собственный репозиторий

Передать доступную проверяющему ссылку и хеш итогового коммита. Включить Assets с .meta, Packages, ProjectSettings, Release, README и .gitignore. Не включать Library, Temp, Logs, obj, UserSettings и Build. Проверить восстановление в новой папке. ZIP образца не является студенческим репозиторием.

2

Единый отчёт

Заполнить «Отчет о выполнении.docx»: 1. Исходные условия; 2. Выполнение и проверка; 3. Итоговая проверка и сдача. Указать исходный и итоговый коммиты, таблицу экранов/размеров, собственные рисунки и фактические результаты T01–T16. Вписать URL репозитория в самом отчёте. Сохранить DOCX и его PDF-копию без изменения структуры.

3

Release и передача

В папке Release сохранить описания, инструкцию игроку, CREDITS, CHANGELOG, иконку, рекламный кадр, реальные Screenshots, макет PRODUCT-PAGE.html и DEMO.md. В README указать запуск, ориентацию, версию и ограничения. Если сделана исполняемая сборка, приложить её через релиз хостинга с платформой. Передать ссылку, хеш и отчёт через установленный канал сдачи; сроки брать из назначения практики.

ПП ПМ.13 · Раздел 6

2. Файлы для работы

↓

Скачивания

Скачать контрольный пример «Кристальный маршрут» (ZIP)

Скачать форму отчёта (DOCX)

Инструмент создания графики (C#)

1

Форма отчёта

Скачать «Отчет о выполнении.docx» и сохранить рабочую копию. Заполнять её собственными данными по ходу этапов; исходный чистый бланк сохранить отдельно.

2

Контрольный пример для сверки

PP13-CrystalRoute-UnityProject.zip — отдельный полный пример, а не индивидуальный стенд. При необходимости распаковать, в Unity Hub выбрать Add → Add project from disk и папку CrystalRoute с Assets, Packages, ProjectSettings. В примере открыть Assets/Scenes/Main.unity. Читать код и сравнивать приём разрешено без повторного выполнения всей игры.

3

Полные файлы C#

Ссылки рядом с листингами открывают полные файлы. Фрагменты в PDF — продолжения одного файла. В существующий индивидуальный проект переносить только нужный приём, сохраняя собственные игровые классы и правило; не заменять весь проект кодом образца. Для графики отдельно приложен редакторский инструмент ReleaseAssets.cs.

4

Один собственный проект

Отдельные стенды вариантам не требуются: входом является готовая собственная игра. Образец показывает выпуск, а не готовое решение всех 30 правил. Если входная игра не соответствует карточке, остановить оформление и восстановить исходную работу; переименование кристаллов не реализует доставку, набор или ворота.

3. Методика на образце

1

Исходное состояние

Образец — «Кристальный маршрут»: три уровня с 3/5/7 кристаллами, 2/4/6 опасностями и временем 90/80/70 секунд. Собрать цель и войти в портал; три жизни. Эти числа служат для чтения примера. В своём проекте записать значения из собственного правила и уровней.

2

Как пользоваться разбором

Найти в своём проекте соответствующий экран и обработчик. Прочитать действие на образце, выполнить нужное изменение в своей игре, затем проверить ожидаемое состояние. Отдельный запуск и сдача образца не требуются. Новые компоненты образца: SafeAreaPanel, PanelTransition, TutorialController, AboutPanel, ScreenshotCapture.

3

Сохранение изменений

Изменения файлов C# сохранять в редакторе до возврата в Unity. Настройки объектов, созданных во время Play, временные: после остановки они исчезают. Постоянные изменения заносить в исходный код, сохранённую сцену или префаб вне Play, затем File → Save / Ctrl+S. PlayerPrefs сохраняет отдельные постоянные данные; не путать его со сценой.

ПП ПМ.13 · Раздел 8

3.1. Входная игра и исходный коммит

1

Что и зачем

Коммит — сохранённое состояние файлов. PlayerPrefs хранится отдельно от Git; копирование репозитория само по себе не создаёт нового игрока.

2

Действия

Открыть собственный проект в Unity Hub. В Project открыть Assets/Scenes/Main.unity либо свою стартовую сцену. Нажать Play, пройти первый уровень и проверить открытие второго. Записать рекорд, доступные уровни и громкость. Остановить Play, сохранить сцену. В терминале папки проекта выполнить git status, затем сохранить завершённое исходное состояние в собственном коммите и получить git rev-parse HEAD.

3

Проверка и доказательство

В отчёте, этап 1, записаны исходный хеш и наблюдаемые данные. До оформления работают победа, поражение и повтор. Для образца первый уровень требует 3 кристалла, а ранний портал сообщает, сколько ещё нужно.

4

Если результат отличается

При утрате проекта восстановить собственный репозиторий через git clone <URL> в новую папку; URL взять из своей карточки репозитория. Если игрового цикла нет, восстановить его до текущих изменений.

3.2. Карта компонентов и перенос

1

Что и зачем

Компонент — скрипт, прикреплённый к объекту Unity. Наличие файла в Assets ещё не запускает его методы.

2

Действия

В примере открыть MenuController.Start и проследить создание Canvas → SafeArea → SafeContent, затем TutorialController, AboutPanel и ScreenshotCapture. В своей игре отметить аналогичные места: создание UI, кнопки меню, обновление HUD и запуск уровня. Добавить SafeAreaPanel и PanelTransition; TutorialController и AboutPanel используют методы Panel/Label/Button образца, поэтому переносить их вместе с нужными UI-методами либо связать действия с уже существующими кнопками. Все полные листинги находятся далее.

3

Проверка и доказательство

После сохранения всего связанного набора Console не содержит красных ошибок. В примере 20 runtime-файлов; это состав образца, а не норма числа файлов индивидуального проекта.

4

Если результат отличается

Не добавлять второй класс с тем же именем. Не заменять собственный GameManager, WorldBuilder, SaveService и LevelManager готовыми файлами: это удалит правило и данные варианта.

3.3. Безопасная область

1

Что и зачем

Screen.safeArea — доступный прямоугольник экрана в пикселях. Anchor — доля родительского прямоугольника от 0 до 1. SafeContent содержит макет 1280×720.

2

Действия

В примере прочитать SafeAreaPanel.Initialize и ApplyRect. В своей игре поместить панели в контейнер безопасной области. Для программно создаваемого UI использовать схему MenuController.Start: создать SafeArea с RectTransform и SafeAreaPanel, вызвать Initialize с RectTransform Canvas и использовать возвращённый SafeContent как родителя панелей. При Play раскрыть Canvas → SafeArea → SafeContent в Hierarchy.

3

Проверка и доказательство

Панели находятся внутри SafeContent. При уменьшении доступной ширины масштаб равен меньшему из width/1280 и height/720: пропорции не искажаются. Изменения во время Play не считать сохранением сцены.

4

Если результат отличается

Если панели остаются прямыми детьми Canvas, безопасный контейнер на них не действует. В образце не добавлять SafeAreaPanel второй раз вручную.

ПП ПМ.13 · Раздел 11

3.4. Имитация вырезов

1

Что и зачем

Simulate Insets — учебные отступы: 8% слева и справа, 5% сверху и снизу. Это проверка макета без телефона.

2

Действия

Запустить Play. При первом обучении нажать «Пропустить» и дождаться меню. В Hierarchy выбрать Canvas → SafeArea; в Inspector включить Simulate Insets. Открыть «Как играть», пройти три карточки, открыть «Об игре», вернуться. Проверить крайние кнопки мышью. Сохранить собственный кадр с Inspector и уменьшенным UI. Выключить флажок и повторить.

3

Проверка и доказательство

Кнопки остаются внутри центральной области и нажимаются; после отключения макет возвращается. Сохранить результат в этапе 2 отчёта. Обычный снимок меню без флажка не доказывает работу имитации.

4

Если результат отличается

Выбрать созданный во время Play объект, а не исходный пустой Canvas. Проверку выреза на настоящем устройстве указывать только после фактического запуска.

3.5. Три размера окна

1

Что и зачем

Game View — окно показа игры. Соотношение сторон определяется шириной и высотой; изменение окна редактора не всегда меняет фиксированное разрешение.

2

Действия

Во вкладке Game раскрыть список разрешений, нажать +, выбрать Fixed Resolution и задать Width 1280, Height 720. Проверить все семь экранов из задания. Повторить для 1440×720 и 1024×768. В отчёте заполнить строки «экран — размер — действие — ожидается — фактически». Сделать по одному читаемому снимку каждого размера и дополнительные снимки найденных обрезаний.

3

Проверка и доказательство

Видны самые длинные надписи, крайние кнопки и всё игровое поле. В WorldBuilder примера размер ортографической камеры — max(6.2, 9/aspect); это видимость уровня, SafeAreaPanel отвечает за UI.

4

Если результат отличается

Если подпись помещается только в широком окне, расширить её область либо перенести текст и повторить все три размера. В своём варианте проверить полный порядок, груз/предел, комплект или названия ворот.

3.6. Переходы панелей на паузе

1

Что и зачем

CanvasGroup управляет прозрачностью и вводом. Time.unscaledDeltaTime продолжает отсчёт при Time.timeScale = 0.

2

Действия

В PanelTransition прочитать OnEnable и Update: alpha начинает с 0, кнопки временно отключены, blocksRaycasts включён. В MenuController.Panel компонент добавляется панелям с окончанием Panel. В своей игре применить переход к аналогичным панелям. Запустить уровень, нажать «Пауза», дождаться появления, нажать «Продолжить». Повторить быстрое нажатие во время появления.

3

Проверка и доказательство

Появление длится 0,18 секунды и заканчивается при паузе. Пока alpha < 1, кнопки этой панели не работают. Закрытие отключает объект через SetActive(false), поэтому он не перехватывает ввод. Статичный снимок показывает только итоговую компоновку.

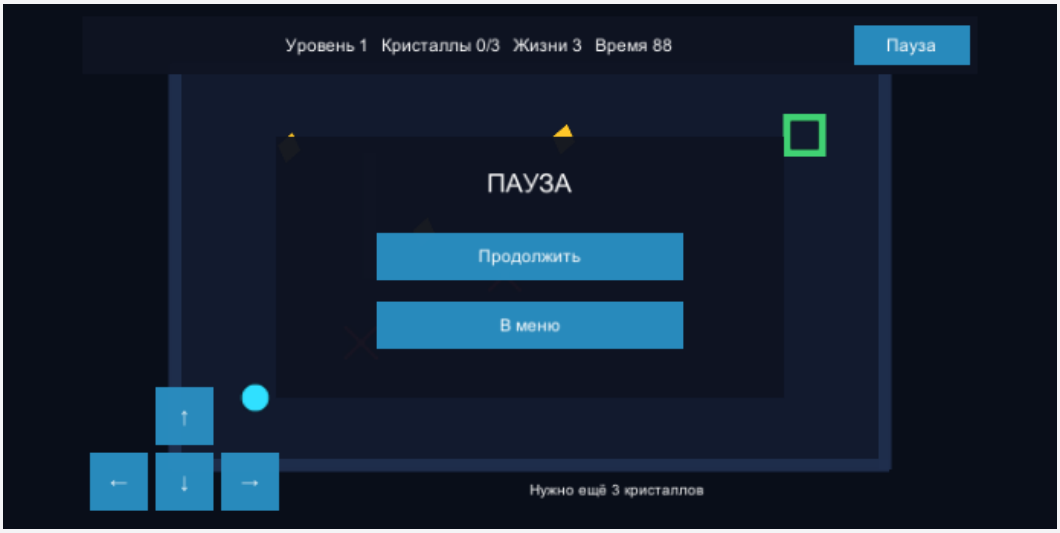
4

Если результат отличается

CanvasGroup блокирует лучи только там, где есть UI Graphic с Raycast Target. В образце это прямоугольник панели, не весь экран. Если под модальным окном остались активные элементы, отключить нижний экран, как menu в Update, либо добавить полноэкранный фон. Не скрывать окно одной прозрачностью 0.

5

Снимок результата



Сохранённый снимок контрольного примера. Конечное состояние панели паузы. Статичный кадр показывает компоновку; длительность перехода проверяется в Play.

ПП ПМ.13 · Раздел 14

3.7. Три текста обучения

1

Что и зачем

Массив steps содержит три строки. Это объяснение до начала игры, а не интерактивная проверка движения.

2

Действия

В TutorialController.cs найти steps. Заменить первую строку своим управлением и действием остановки. Вторую посвятить цели и правилу карточки; третью — ошибке, последствиям и условию выхода. Сохранить кавычки, запятые и символы \n для переносов. В HUD своей игры использовать те же названия предметов, счётчиков и зон. Нажать Play и прочитать каждую карточку.

3

Проверка и доказательство

Подсказки называют действие и наблюдаемый результат. Для доставки явно различаются подбор и зачёт; для набора — отдельные типы. Сохранить три собственных текста в этапе 3 отчёта.

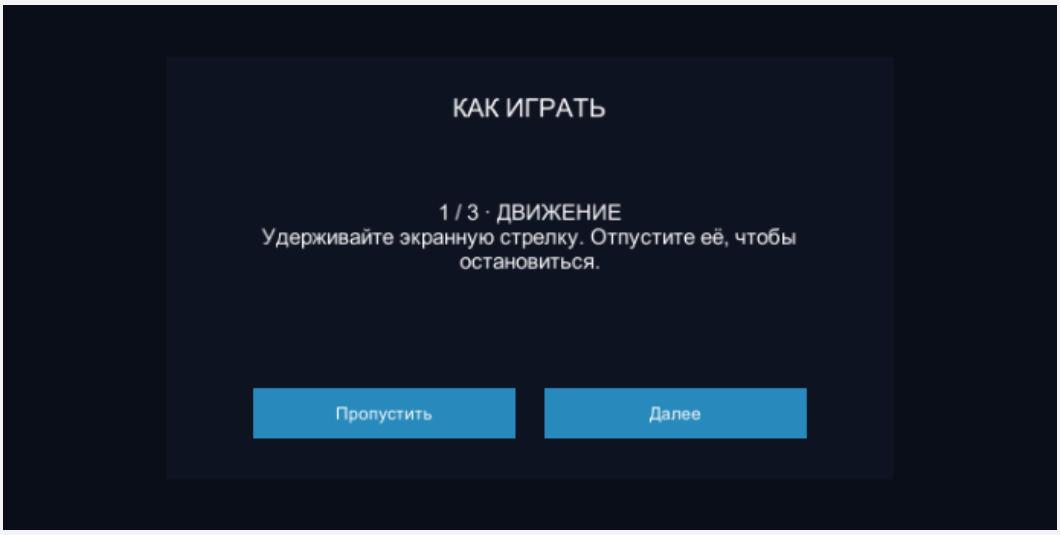
4

Если результат отличается

Не переносить фразу «собрать всё» в вариант с запретными предметами или вместимостью. Порядок требует пояснения неверного шага и сброса, если сброс предусмотрен карточкой.

5

Снимок результата



Сохранённый снимок контрольного примера. Первая карточка объясняет удержание и отпускание экранной стрелки.

ПП ПМ.13 · Раздел 15

3.8. Далее, пропуск и возврат

1

Что и зачем

index — индекс массива, начинается с 0. Advance меняет индекс, а на последней карточке вызывает Close.

2

Действия

Открыть «Как играть». Прочитать карточку 1, нажать «Далее», проверить карточку 2, ещё раз «Далее» — карточку 3. Убедиться, что последняя кнопка называется «В меню». Нажать её. Открыть обучение заново и проверить «Пропустить» на первой, затем на второй карточке.

3

Проверка и доказательство

После третьей карточки или пропуска открывается меню; карточки 4 нет. Повтор всегда начинается с карточки 1. В отчёте сохранить последнюю карточку и возвращённое меню.

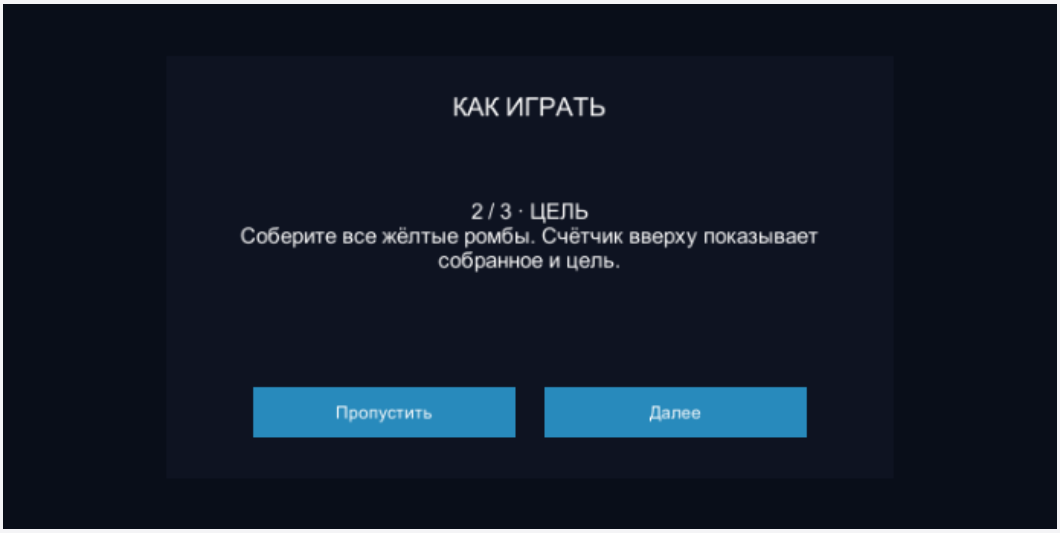
4

Если результат отличается

Индекс должен увеличиваться только при index < steps.Length - 1. У закрытой панели IsOpen=false, повторное событие не должно продвигать скрытую карточку.

5

Снимок результата



Сохранённый снимок контрольного примера. Вторая карточка объясняет цель и счётчик. Следующая карточка посвящена опасности и выходу.

ПП ПМ.13 · Раздел 16

3.9. Первый запуск и сохранения

1

Что и зачем

SeenKey в образце равен PP13.Crystal.Tutorial.v1. Игровые ключи в SaveService имеют другой префикс; это разные записи.

2

Действия

Перед изменением записать исходные рекорд, доступ и громкость. Для отдельной проверки нового обучения в рабочей копии изменить только строку SeenKey на уникальный ключ своей игры с суффиксом Tutorial.v1, не трогая SaveService и Company/Product Name. Нажать Play: появится карточка 1. Завершить или пропустить обучение, остановить и снова включить Play. Затем открыть «Как играть».

3

Проверка и доказательство

Автоматически показывается меню, ручной повтор начинается с 1, рекорды, доступ и громкость прежние. После полного закрытия и повторного открытия редактора проверить ещё раз; новый Play и полный перезапуск — отдельные строки проверки.

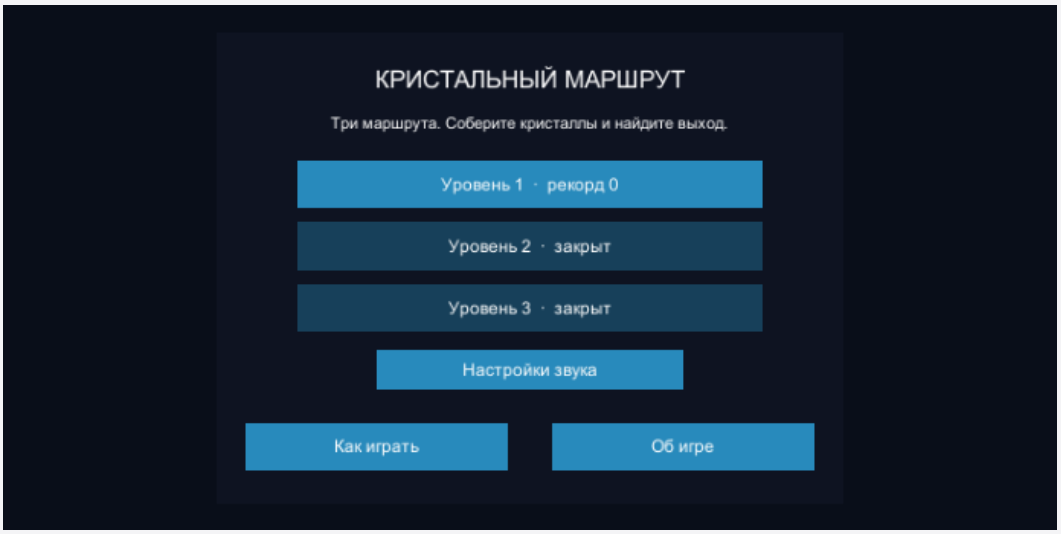
4

Если результат отличается

Не применять PlayerPrefs.DeleteAll. Изменение только ключа обучения не проверяет нового игрока по игровому прогрессу. Установленный ключ оставить постоянным; менять его в каждом запуске нельзя.

5

Снимок результата



Сохранённый снимок контрольного примера. В меню доступны «Как играть» и «Об игре». Закрытые уровни сохраняют прежнее поведение.

3.10. Сведения об игре

1

Что и зачем

Application.version читает Version из Player Settings. Авторство описывает фактических создателей игры и ресурсов.

2

Действия

Открыть Edit → Project Settings → Player, установить Version 1.0.0 для первого учебного выпуска. В AboutPanel заменить название и сведения об авторах и ресурсах. Создать Release/CREDITS.md. Запустить Play, в меню открыть «Об игре», сверить Version, вернуться кнопкой «Назад». В своей реализации связать кнопку открытия с AboutPanel.Open только в состоянии Menu.

3

Проверка и доказательство

Название относится к индивидуальному проекту; номер совпадает на экране, в Player Settings и README. Источники ресурсов доступны в Release/CREDITS.md.

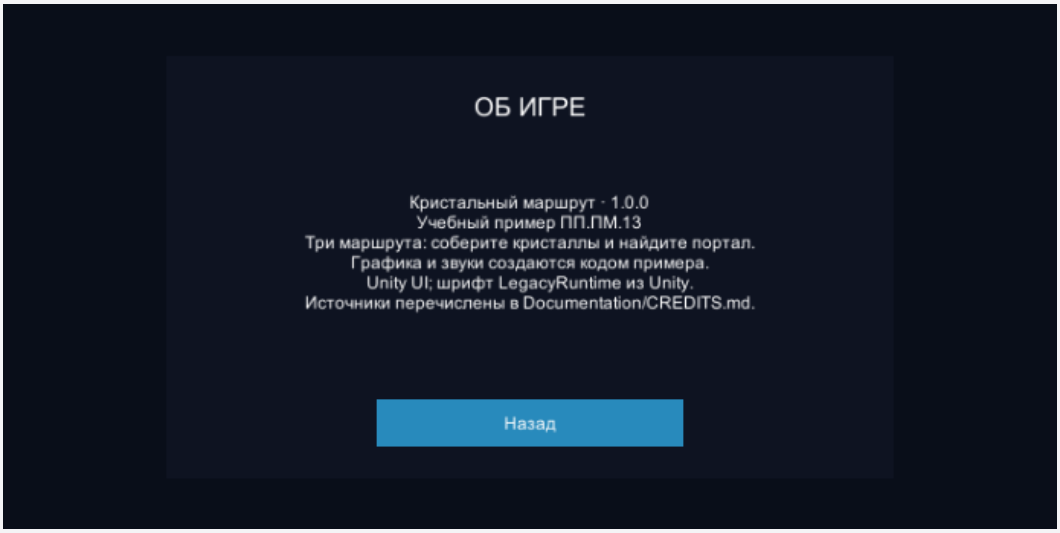
4

Если результат отличается

Сохранить прежние Company Name и Product Name при проверке существующего прогресса: они влияют на расположение данных. Не оставлять чужие имена или неустановленную лицензию.

5

Снимок результата



Сохранённый снимок компоновки экрана «Об игре» контрольного примера. Сведения о ресурсах своей игры разместить в Release/CREDITS.md.

ПП ПМ.13 · Раздел 18

3.11. PNG текущего экрана

1

Что и зачем

F9 вызывает ScreenshotCapture.Capture. WaitForEndOfFrame ждёт конец отрисовки; Debug.Log сообщает запрос записи, а не доказанную готовность PNG.

2

Действия

Открыть нужный экран в Game, дождаться перехода, нажать F9 (при необходимости Fn+F9). В Window → General → Console найти Screenshot requested и полный путь. Открыть указанный каталог через проводник; найти PNG с текущим временем, открыть его и проверить содержимое. При AppData, скрытой в проводнике, вставить полный путь из Console в адресную строку.

3

Проверка и доказательство

PNG содержит текущий Game View. Файл находится в Screenshots внутри Application.persistentDataPath, а не автоматически в папке Release. Для отчёта сохранить путь и реальный открытый файл.

4

Если результат отличается

При переходе из Game в Scene ожидание конца кадра может задержаться; вернуть фокус Game. Если PNG не появился, проверить ошибки доступа и повторить захват после устранения причины. Не считать сообщение об успешном запросе готовым файлом.

ПП ПМ.13 · Раздел 19

3.12. Шесть состояний для представления

1

Что и зачем

Публикационные кадры показывают фактическую игру; их нельзя заменять рисунком интерфейса.

2

Действия

Установить Game View 1280×720 и отключить Simulate Insets. Последовательно сохранить F9: меню, обучение, игровой процесс с собственным правилом, паузу, результат победы, «Об игре». В проводнике создать Release/Screenshots в корне своего проекта и скопировать PNG как menu.png, tutorial.png, gameplay.png, pause.png, result.png, about.png. Открыть каждый файл.

3

Проверка и доказательство

Шесть разных кадров имеют одинаковый размер и читаемый UI. Скриншоты условий вариантов дополнительно показывают состояние до и после действия. Исторические снимки образца служат иллюстрациями, собственные результаты ими не заменять.

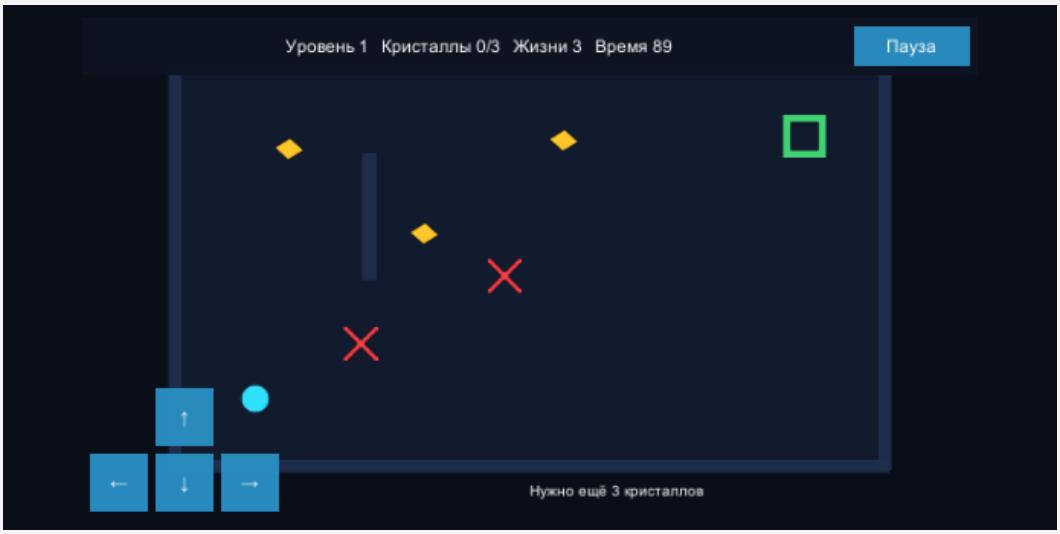
4

Если результат отличается

Значения рекорда и таймера могут отличаться от образца. Не показывать отладочную победу как прохождение игроком; не растягивать изображение до другого соотношения сторон.

5

Снимок результата



Сохранённый снимок контрольного примера. Реальный кадр первого уровня контрольного примера: игровой процесс и HUD.

ПП ПМ.13 · Раздел 20

3.13. Иконка в том же проекте

1

Что и зачем

Редакторский инструмент ReleaseAssets создаёт графику камерой в preview scene, не запускает игровой цикл и не меняет Main. Отдельная игра для графики не нужна.

2

Действия

Если инструмент отсутствует, создать в Assets папку Editor и поместить туда предоставленный полный ReleaseAssets.cs. Дождаться компиляции. В редакторе C# найти создание symbol и подписи в методе Export. В рабочей копии заменить подписи названием и свойствами своей игры, цвет — собственным акцентом. Сохранить файл. При выключенном Play выбрать Practice → Export Release Artwork.

3

Проверка и доказательство

В корне проекта создаются Release/icon.png 512×512 и Release/promo.png 1280×720. Открыть их в проводнике. Крупный символ различим в уменьшенном виде; подписи относятся к своей игре.

4

Если результат отличается

ReleaseAssets.cs должен находиться в Editor: он использует UnityEditor и не входит в приложение. Если меню Practice не появилось, сначала устранить красные ошибки Console. Команда перезаписывает эти два выходных файла: собственные прежние картинки предварительно скопировать.

ПП ПМ.13 · Раздел 21

3.14. Символ и рекламная композиция

1

Что и зачем

Иконка и promo — рекламная графика, а не доказательство прохождения. Изменить композицию разрешено; обещания в тексте обязаны совпадать с игрой.

2

Действия

В ReleaseAssets.Export первая Save сохраняет квадрат, вторая — широкий кадр. Для простого своего символа выбрать форму PrimitiveType.Cube, Sphere либо исходный Quad; положение и масштаб заданы transform. Для сочетания деталей повторить создание symbol с другим именем переменной и переместить второй объект в ту же preview scene. Для текста использовать вызов Text с нужной строкой и размером. Повторить Export, открыть файлы.

3

Проверка и доказательство

Иконка не содержит мелких нечитаемых деталей. В рекламном изображении остаётся место названию. Размеры проверить через свойства PNG. Готовая графика помещена в Release.

4

Если результат отличается

Не добавлять GameBootstrap в preview scene и не запускать пустую Icon через Play: загрузчик образца создаёт игру в любой сцене. Процедура экспорта работает в редакторе без Play; отдельно восстанавливать Main после неё не требуется.

ПП ПМ.13 · Раздел 22

3.15. Статичный макет страницы

1

Что и зачем

HTML — текстовый файл страницы, открываемый браузером локально. PRODUCT-PAGE.html образца показывает расположение материалов и не отправляет данные в магазин.

2

Действия

Скопировать предоставленный PRODUCT-PAGE.html в Release своей игры. Открыть в редакторе текста. Заменить название, версию, авторов, краткое и полное описания. Сохранить рядом icon.png, promo.png и папку Screenshots с указанными именами. Открыть HTML двойным щелчком в браузере; проверить все картинки и тексты, затем сохранить собственный снимок макета.

3

Проверка и доказательство

Страница содержит название, иконку, реальные кадры, два описания, версию и авторов. После копирования Release в новую папку изображения по относительным путям продолжают открываться.

4

Если результат отличается

Не оставлять тексты образца. При значке потерянной картинки сверить регистр и имя файла в src; браузер читает его относительно HTML. Локальный макет не является опубликованной страницей магазина.

ПП ПМ.13 · Раздел 23

3.16. Описания и инструкция игроку

1

Что и зачем

Markdown — обычный текст с заголовками. Разделить обещание до запуска и конкретную помощь игроку.

2

Действия

В Release создать DESCRIPTION.md. Написать краткое описание до 80 знаков, затем полное 600–1200 знаков. Проверить выделенный фрагмент через рецензирование/статистику Word; считать знаки с пробелами, без заголовков. В PLAYER_GUIDE.md описать управление, остановку, цель, ошибку, паузу, повтор и сохранение. Использовать собственные предметы и правило карточки.

3

Проверка и доказательство

Каждое утверждение можно сопоставить с кадром или проверкой. Полное описание не обрезано в HTML; локальные сохранения не названы облачными.

4

Если результат отличается

Не обещать сеть, рекламу, мобильный запуск или количество уровней, которых нет. В файлах Release образца приведены примеры структуры; их текст не становится готовым описанием индивидуального варианта.

ПП ПМ.13 · Раздел 24

3.17. Проверка понятности

1

Что и зачем

Наблюдать конкретные действия участника, а не только мнение «нравится». Подготовить чистый ключ обучения, сохранив игровой прогресс.

2

Действия

Попросить начать игру, объяснить особое правило по подсказкам, выполнить его, поставить паузу и продолжить. Во время попытки не помогать. Записать место остановки, слова или действие, затем уточнить непонятное. Изменить конкретную подпись/подсказку и повторить тот же сценарий. Если участника нет, выполнить самостоятельный проход и указать это в отчёте.

3

Проверка и доказательство

В этапе 5 отчёта отделены исходное затруднение, изменение и повторный результат. Отсутствие внешнего участника явно отмечено как ограничение, а не успешный пользовательский тест.

4

Если результат отличается

Не придумывать имя, цитату, время или результат. Для повторной проверки оставить сопоставимые начальные условия. Проверить, отличает ли человек подбор от доставки, сумму от набора, один переключатель от двух разных.

ПП ПМ.13 · Раздел 25

3.18. Источники и изменения выпуска

1

Что и зачем

CREDITS отвечает, откуда ресурсы; CHANGELOG — что изменилось в продукте. Это разные файлы.

2

Действия

В CREDITS.md перечислить каждый сторонний ресурс: имя, автор, точная ссылка, условия использования. Собственные ресурсы указать отдельно. Для встроенного Unity UI/шрифта написать происхождение, не объявлять их собственными. В CHANGELOG.md указать 1.0.0, функции выпуска и известные ограничения. Сверить эти сведения с AboutPanel и README.

3

Проверка и доказательство

На экране и в файлах нет противоречий. Отдельно отмечены действительно собственные изображения и звуки. Неизвестные права не заменены вымышленной свободной лицензией.

4

Если результат отличается

Если происхождение ресурса не установлено, заменить его собственным и обновить список. Историю разработки методички в CHANGELOG игры не добавлять.

ПП ПМ.13 · Раздел 26

3.19. Сценарий 60–90 секунд

1

Что и зачем

Раскадровка связывает кадр с проверяемой возможностью; это доступная замена видео.

2

Действия

Создать Release/DEMO.md: 0–10 с — меню; 10–25 — обучение; 25–45 — особое правило; 45–55 — результат; 55–65 — повтор; 65–75 — «Об игре». Для каждого интервала добавить относительную ссылку на один из 6–8 подготовленных PNG и подпись о действии. Для повтора снять отдельный кадр начала новой попытки. Пройти сценарий с секундомером.

3

Проверка и доказательство

Длительности складываются в 75 секунд и находятся в заданном диапазоне. В кадрах видны собственное правило, победа и повтор. При наличии реальной видеозаписи можно приложить её дополнительно.

4

Если результат отличается

Шесть одинаковых снимков меню не демонстрируют игру. Кадр результата не заменяет видимый особый приём. Отсутствующий ролик не описывать как снятый.

ПП ПМ.13 · Раздел 27

3.20. Сборка при наличии модуля

1

Что и зачем

Build Profile задаёт платформу и сцены. Основная сдача — Unity-проект; возможность получить приложение зависит от установленного модуля.

2

Действия

Открыть File → Build Profiles. Выбрать доступную платформу, в списке сцен оставить Main либо собственную стартовую сцену и нужные игровые сцены. Сцены подготовки графики не включать. Сверить Version и название. Если модуль уже установлен, выполнить Build в отдельную папку вне исходников, запустить приложение и повторить основные проверки. Если модуля нет, записать границу в README.

3

Проверка и доказательство

Либо есть фактически запущенная сборка с указанной платформой, либо честно передаётся проект без исполняемого файла. Android не объявлен проверенным по Game View.

4

Если результат отличается

Не устанавливать дополнительные модули ради необязательного пути. Build не заменяет Assets/Packages/ProjectSettings; файл приложения при наличии распространяется отдельно через релиз хостинга.

ПП ПМ.13 · Раздел 28

3.21. T01–T16 и регрессия

1

Что и зачем

Регрессия — повторная проверка игрового цикла после изменения интерфейса. Зелёная Console не подтверждает понятность и сохранения.

2

Действия

Выполнить T01–T16. В таблице отчёта для каждого теста указать исходные данные, действие, ожидаемое и фактическое. Пройти все три уровня, ранний выход, потерю жизней/времени, поражение и повтор. Сверить индивидуальное правило по карточке. Проверить рекорды, доступ, громкость и обучение после нового Play и полного перезапуска. Открыть каждый файл Release.

3

Проверка и доказательство

Наблюдения записаны по отдельным случаям. У образца после третьего уровня нет четвёртого; поражение не открывает следующий. Непроверенная платформа или отсутствие участника отмечены.

4

Если результат отличается

Не переносить ожидаемые числа образца в свой вариант. После найденного расхождения исправить причину и повторить затронутый тест вместе с соседними состояниями.

ПП ПМ.13 · Раздел 29

3.22. Отчёт и передача одного проекта

1

Что и зачем

Хеш фиксирует сдаваемую версию исходников. Отчёт содержит собственные данные и доказательства, а не утверждение о готовности.

2

Действия

В чистой копии «Отчет о выполнении.docx» заполнить три раздела и поля этапов, заменить подсказки собственными рисунками. Обновить поля Ctrl+A → F9, проверить подписи и ссылки. В самом отчёте указать URL своего репозитория. Сохранить DOCX и экспортировать PDF через Word. В проекте проверить .gitignore, сохранить завершённые изменения в коммите, отправить и записать git rev-parse HEAD. Склонировать URL в новую папку, открыть проект, выполнить T16; итоговый хеш и результат внести в отчёт.

3

Проверка и доказательство

Проверяющему доступны проект и Release нужного коммита; обе формы отчёта открываются. В отчёте указаны исходный и итоговый хеш, результаты и ограничения. Передать URL, хеш и отчёт через установленный канал.

4

Если результат отличается

Не коммитить Library/Temp/Logs/obj/UserSettings/Build. Если в новой копии отсутствуют материалы или доступ закрыт, исправить передачу и проверить заново. Публичность репозитория не обязательна, доступ проверяющему обязателен.

ПП ПМ.13 · Раздел 30

4. Перенос · Выборочный сбор

1

Действия и проверяемая логика

Для двух типов хранить отдельные подписи и последствия. В steps[1] назвать верный и неверный предмет. В обработчике неверного контакта менять только предусмотренный ресурс, не общий счёт цели. В HUD отображать правильный сбор и ошибки/заряд. При ошибке выводить причину и изменившееся значение. До действия записать ресурс R; после одного контакта сравнить с R минус установленный штраф; правильная цель не увеличилась.

2

Сохранение своей игры

Использовать названия классов и данных своего проекта. Сначала сопоставить правило карточки с готовой реализацией, затем добавить объяснение и отображение. Новый учебный интерфейс не меняет победу, наказание, три уровня и постоянные данные. Сохранить состояния до и после в этапе 5 отчёта.

4. Перенос · Порядок

1

Действия и проверяемая логика

Хранить список ожидаемых элементов и index следующего. В steps[1] и HUD показывать next = sequence[index]. В неверной ветви сохранить предусмотренное карточкой поведение: сброс к началу либо отказ без продвижения. При сбросе восстановить доступность уже использованных объектов; при отказе оставить index прежним. Проверить второй элемент первым, правильный первый, повтор первого и новую попытку. Не объяснять сброс в игре, которая его не выполняет.

2

Сохранение своей игры

Использовать названия классов и данных своего проекта. Сначала сопоставить правило карточки с готовой реализацией, затем добавить объяснение и отображение. Новый учебный интерфейс не меняет победу, наказание, три уровня и постоянные данные. Сохранить состояния до и после в этапе 5 отчёта.

ПП ПМ.13 · Раздел 32

4. Перенос · Доставка

1

Действия и проверяемая логика

Хранить переносимый объект отдельно от delivered. При подборе менять carried, при входе в верную зону увеличивать delivered один раз и очищать carried. HUD различает «нет груза», «груз взят», «доставлено». В подсказке назвать конкретную зону и момент зачёта. Проверить вход пустым, подбор, неверную зону, верную доставку и повторный вход. Урон меняет груз только по правилу входной игры.

2

Сохранение своей игры

Использовать названия классов и данных своего проекта. Сначала сопоставить правило карточки с готовой реализацией, затем добавить объяснение и отображение. Новый учебный интерфейс не меняет победу, наказание, три уровня и постоянные данные. Сохранить состояния до и после в этапе 5 отчёта.

ПП ПМ.13 · Раздел 33

4. Перенос · Вместимость

1

Действия и проверяемая логика

Использовать carried, capacity и delivered. Подбор разрешён при carried < capacity; предмет сверх лимита остаётся на поле. Разгрузка добавляет фактический carried к delivered и обнуляет carried. Подсказка при полном грузе указывает место разгрузки. Проверить предел C, отказ C+1, частичную разгрузку, пустую разгрузку и повтор: слагаемое не должно учитываться дважды.

2

Сохранение своей игры

Использовать названия классов и данных своего проекта. Сначала сопоставить правило карточки с готовой реализацией, затем добавить объяснение и отображение. Новый учебный интерфейс не меняет победу, наказание, три уровня и постоянные данные. Сохранить состояния до и после в этапе 5 отчёта.

4. Перенос · Комплект

1

Действия и проверяемая логика

Хранить число или флаг отдельно для каждого типа. Условие завершения — все требуемые значения, а не сумма. В подсказке и HUD подписать каждый тип и норму. При избытке одного типа не закрывать отсутствие другого. Проверить состояние с достаточной общей суммой, но без одного компонента, затем полный набор и повтор попытки.

2

Сохранение своей игры

Использовать названия классов и данных своего проекта. Сначала сопоставить правило карточки с готовой реализацией, затем добавить объяснение и отображение. Новый учебный интерфейс не меняет победу, наказание, три уровня и постоянные данные. Сохранить состояния до и после в этапе 5 отчёта.

4. Перенос · Доступ

1

Действия и проверяемая логика

Хранить флаг каждого активатора и соответствие проходу. При активации менять связанный проход: выключить его Collider либо объект блокировки. Для двух рубильников проверять два разных флага через AND, повтор одного не меняет второй. В UI показать символ и название соответствующего прохода. Проверить чужой ключ, повтор активатора, неполное и полное условие, физическое прохождение и восстановление при повторе.

2

Сохранение своей игры

Использовать названия классов и данных своего проекта. Сначала сопоставить правило карточки с готовой реализацией, затем добавить объяснение и отображение. Новый учебный интерфейс не меняет победу, наказание, три уровня и постоянные данные. Сохранить состояния до и после в этапе 5 отчёта.

ПП ПМ.13 · Раздел 36

Код · AboutPanel.cs

1

Назначение и вставка

Новый компонент. Создаёт экран названия, версии и авторства, чтобы сведения о выпуске были доступны из меню. MenuController добавляет компонент и вызывает Initialize.
Полная версия AboutPanel.cs приведена далее по частям. Единица использования — весь одноимённый файл; заголовки и номера строк в него не вставлять. В своём проекте читать способ и переносить нужный приём, сохраняя собственную игровую логику.

ПП ПМ.13 · Раздел 37

AboutPanel.cs · строки 1–28

1

Продолжение полного файла

```
using UnityEngine;

public class AboutPanel : MonoBehaviour
{
    private GameObject panel;
    public bool IsOpen => panel != null && panel.activeSelf;

    public void Initialize(MenuController ui, Transform parent)
    {
        panel = ui.Panel("AboutPanel", parent, 0, 0, 1000, 580);
        ui.Label(panel.transform, "ОБ ИГРЕ", 0, 220, 900, 60, 36);
        ui.Label(panel.transform, "Кристальный маршрут · " + Application.version +
            "\nУчебный пример ПП.ПМ.13\n" +
            "Три маршрута: соберите кристаллы и найдите портал.\n" +
            "Графика и звуки создаются кодом примера.\n" +
            "Unity UI; шрифт LegacyRuntime из Unity.\n" +
            "Источники перечислены в Release/CREDITS.md.",
            0, 20, 900, 300, 25);
        ui.Button(panel.transform, "AboutBack", "Назад", 0, -215,
            420, 65, () => panel.SetActive(false));
        panel.SetActive(false);
    }

    public void Open()
    {
        if (GameManager.Instance.CurrentState != GameManager.State.Menu) return;
        panel.SetActive(true);
        panel.transform.SetAsLastSibling();
```

ПП ПМ.13 · Раздел 38

AboutPanel.cs · строки 29–30

1

Продолжение полного файла

```
    }
}
```

ПП ПМ.13 · Раздел 39

Код · AudioFeedback.cs

1

Назначение и вставка

Входная часть контрольного примера. Воспроизводит звуковую обратную связь на игровые события. Здесь приведён для полноты рабочего примера; повторно добавлять такой компонент не нужно.
Полная версия AudioFeedback.cs приведена далее по частям. Единица использования — весь одноимённый файл; заголовки и номера строк в него не вставлять. В своём проекте читать способ и переносить нужный приём, сохраняя собственную игровую логику.

AudioFeedback.cs · строки 1–28

1

Продолжение полного файла

```
using UnityEngine;

[RequireComponent(typeof(AudioSource))]
public class AudioFeedback : MonoBehaviour
{
    private AudioSource source;
    private AudioClip collectClip;
    private AudioClip hazardClip;
    private AudioClip successClip;

    private void Awake()
    {
        EnsureReady();
    }

    private void EnsureReady()
    {
        if (source == null)
        {
            source = GetComponent<AudioSource>();
            source.playOnAwake = false;
        }

        collectClip ??= CreateTone("Collect", 660f, 0.10f, 0.20f);
        hazardClip ??= CreateTone("Hazard", 180f, 0.18f, 0.24f);
        successClip ??= CreateTone("Success", 880f, 0.30f, 0.18f);
    }
```

AudioFeedback.cs · строки 29–56

1

Продолжение полного файла

```
public void PlayCollect()
{
    EnsureReady();
    source.PlayOneShot(collectClip);
}

public void PlayHazard()
{
    EnsureReady();
    source.PlayOneShot(hazardClip);
}

public void PlaySuccess()
{
    EnsureReady();
    source.PlayOneShot(successClip);
}

private static AudioClip CreateTone(string clipName, float frequency,
    float duration, float volume)
{
    const int sampleRate = 44100;
    int sampleCount = Mathf.CeilToInt(sampleRate * duration);
    float[] samples = new float[sampleCount];

    for (int i = 0; i < sampleCount; i++)
    {
        float time = (float)i / sampleRate;
```

ПП ПМ.13 · Раздел 42

AudioFeedback.cs · строки 57–67

1

Продолжение полного файла

```
            float fade = 1f - (float)i / sampleCount;
            samples[i] = Mathf.Sin(2f * Mathf.PI * frequency * time)
                * volume * fade;
        }

        AudioClip clip = AudioClip.Create(clipName, sampleCount, 1,
            sampleRate, false);
        clip.SetData(samples, 0);
        return clip;
    }
}
```

ПП ПМ.13 · Раздел 43

Код · Collectible.cs

1

Назначение и вставка

Входная часть контрольного примера. Обрабатывает взаимодействие игрока с собираемым предметом. Здесь приведён для полноты рабочего примера; повторно добавлять такой компонент не нужно.
Полная версия Collectible.cs приведена далее по частям. Единица использования — весь одноимённый файл; заголовки и номера строк в него не вставлять. В своём проекте читать способ и переносить нужный приём, сохраняя собственную игровую логику.

ПП ПМ.13 · Раздел 44

Collectible.cs · строки 1–16

1

Продолжение полного файла

```
using UnityEngine;

[RequireComponent(typeof(Collider2D))]
public class Collectible : MonoBehaviour
{
    private void Awake()
    {
        GetComponent<Collider2D>().isTrigger = true;
    }

    private void OnTriggerEnter2D(Collider2D other)
    {
        if (!other.TryGetComponent<PlayerController>(out _)) return;
        GameManager.Instance.Collect(this);
    }
}
```

ПП ПМ.13 · Раздел 45

Код · CollectibleVisual.cs

1

Назначение и вставка

Входная часть контрольного примера. Задаёт визуальное поведение собираемого предмета. Здесь приведён для полноты рабочего примера; повторно добавлять такой компонент не нужно.
Полная версия CollectibleVisual.cs приведена далее по частям. Единица использования — весь одноимённый файл; заголовки и номера строк в него не вставлять. В своём проекте читать способ и переносить нужный приём, сохраняя собственную игровую логику.

CollectibleVisual.cs · строки 1–27

1

Продолжение полного файла

```
using UnityEngine;

public class CollectibleVisual : MonoBehaviour
{
    [SerializeField, Min(0f)] private float rotationSpeed = 70f;
    [SerializeField, Range(0f, 0.25f)] private float pulseAmount = 0.08f;
    [SerializeField, Min(0.1f)] private float pulseSpeed = 4f;

    private Vector3 startScale;

    private void Awake()
    {
        startScale = transform.localScale;
    }

    private void Update()
    {
        transform.Rotate(0f, 0f, rotationSpeed * Time.deltaTime);
        float pulse = 1f + Mathf.Sin(Time.time * pulseSpeed) * pulseAmount;
        transform.localScale = startScale * pulse;
    }

    private void OnDisable()
    {
        transform.localScale = startScale;
    }
}
```

ПП ПМ.13 · Раздел 47

Код · ExitZone.cs

1

Назначение и вставка

Входная часть контрольного примера. Обрабатывает вход игрока в зону завершения уровня. Здесь приведён для полноты рабочего примера; повторно добавлять такой компонент не нужно.
Полная версия ExitZone.cs приведена далее по частям. Единица использования — весь одноимённый файл; заголовки и номера строк в него не вставлять. В своём проекте читать способ и переносить нужный приём, сохраняя собственную игровую логику.

ПП ПМ.13 · Раздел 48

ExitZone.cs · строки 1–16

1

Продолжение полного файла

```
using UnityEngine;

[RequireComponent(typeof(Collider2D))]
public class ExitZone : MonoBehaviour
{
    private void Awake()
    {
        GetComponent<Collider2D>().isTrigger = true;
    }

    private void OnTriggerEnter2D(Collider2D other)
    {
        if (!other.TryGetComponent<PlayerController>(out _)) return;
        GameManager.Instance.TryFinishLevel();
    }
}
```

ПП ПМ.13 · Раздел 49

Код · GameBootstrap.cs

1

Назначение и вставка

Входная часть контрольного примера. Создаёт стартовые объекты и компоненты игры. Здесь приведён для полноты рабочего примера; повторно добавлять такой компонент не нужно.
Полная версия GameBootstrap.cs приведена далее по частям. Единица использования — весь одноимённый файл; заголовки и номера строк в него не вставлять. В своём проекте читать способ и переносить нужный приём, сохраняя собственную игровую логику.

ПП ПМ.13 · Раздел 50

GameBootstrap.cs · строки 1–14

1

Продолжение полного файла

```
using UnityEngine;

public static class GameBootstrap
{
    [RuntimeInitializeOnLoadMethod(RuntimeInitializeLoadType.AfterSceneLoad)]
    private static void CreateGame()
    {
        if (Object.FindAnyObjectByType<GameManager>() != null)
            return;

        var root = new GameObject("CrystalQuest");
        root.AddComponent<GameManager>();
    }
}
```

Код · GameManager.cs

1

Назначение и вставка

Входная часть контрольного примера. Управляет состояниями игры и основным игровым циклом. Здесь приведён для полноты рабочего примера; повторно добавлять такой компонент не нужно.
Полная версия GameManager.cs приведена далее по частям. Единица использования — весь одноимённый файл; заголовки и номера строк в него не вставлять. В своём проекте читать способ и переносить нужный приём, сохраняя собственную игровую логику.

GameManager.cs · строки 1–28

1

Продолжение полного файла

```
using UnityEngine;

[DefaultExecutionOrder(-50)]
[RequireComponent(typeof(LevelManager), typeof(WorldBuilder))]
public class GameManager : MonoBehaviour
{
    public enum State { Menu, Playing, Paused, Won, Lost }
    public static GameManager Instance { get; private set; }
    public State CurrentState { get; private set; } = State.Menu;
    public bool IsPlaying => CurrentState == State.Playing;
    public Vector2 MobileDirection { get; private set; }
    public int Collected { get; private set; }
    public int Hearts { get; private set; }
    public float Remaining { get; private set; }
    public string Hint { get; private set; } = "Выберите уровень";
    public LevelManager Levels { get; private set; }
    private WorldBuilder world;
    private AudioFeedback audioFeedback;
    private float immuneUntil;
    private float elapsed;

    private void Awake()
    {
        Instance = this;
        Levels = GetComponent<LevelManager>();
        world = GetComponent<WorldBuilder>();
        audioFeedback = gameObject.AddComponent<AudioFeedback>();
        gameObject.AddComponent<PauseController>();
```

GameManager.cs · строки 29–56

1

Продолжение полного файла

```
        gameObject.AddComponent<MenuController>();
        Application.targetFrameRate = 60;
        AudioListener.volume = SaveService.Volume;
        Time.timeScale = 1;
        world.BuildCamera();
    }

    public void StartLevel(int number)
    {
        if (!Levels.Select(number)) return;
        Time.timeScale = 1;
        Collected = 0;
        Hearts = 3;
        Remaining = Levels.Data.seconds;
        elapsed = 0;
        immuneUntil = 0;
        MobileDirection = Vector2.zero;
        Hint = "Соберите все кристаллы и войдите в портал";
        world.Build(Levels.Data);
        CurrentState = State.Playing;
    }

    private void Update()
    {
        if (!IsPlaying) return;
        elapsed += Time.deltaTime;
        Remaining = Mathf.Max(0, Remaining - Time.deltaTime);
        if (Remaining <= 0) Finish(false, "Время закончилось");
```

GameManager.cs · строки 57–84

1

Продолжение полного файла

```
}

public void SetDirection(Vector2 value) => MobileDirection = value;

public void Collect(Collectible item)
{
    if (!IsPlaying || !item.gameObject.activeSelf) return;
    item.gameObject.SetActive(false);
    Collected++;
    audioFeedback.PlayCollect();
    Hint = Collected == Levels.Data.target
        ? "Все кристаллы собраны. Войдите в зелёный портал"
        : $"Собрано {Collected} из {Levels.Data.target}";
}

public void HitHazard()
{
    if (!IsPlaying || Time.time < immuneUntil) return;
    Hearts--;
    immuneUntil = Time.time + 1;
    audioFeedback.PlayHazard();
    world.Player.ReturnToStart();
    Hint = "Потеряна жизнь. Измените маршрут";
    if (Hearts <= 0) Finish(false, "Жизни закончились");
}

public void TryFinishLevel()
{
```

GameManager.cs · строки 85–112

1

Продолжение полного файла

```
        if (!IsPlaying) return;
        if (Collected < Levels.Data.target)
        {
            Hint = $"Нужно ещё {Levels.Data.target - Collected} кристаллов";
            return;
        }
        int score = Mathf.Max(1, 1000 + Hearts * 100 -
            Mathf.RoundToInt(elapsed * 5));
        SaveService.Complete(Levels.Current, score);
        audioFeedback.PlaySuccess();
        Finish(true, $"Очки: {score}. Рекорд: {SaveService.Best(Levels.Current)}");
    }

    private void Finish(bool won, string message)
    {
        CurrentState = won ? State.Won : State.Lost;
        Hint = message;
        MobileDirection = Vector2.zero;
        Time.timeScale = 0;
    }

    public void Pause()
    {
        if (!IsPlaying) return;
        CurrentState = State.Paused;
        MobileDirection = Vector2.zero;
        Time.timeScale = 0;
        PlayerPrefs.Save();
```

ПП ПМ.13 · Раздел 56

GameManager.cs · строки 113–139

1

Продолжение полного файла

```
    }

    public void TogglePause()
    {
        if (IsPlaying) Pause();
        else if (CurrentState == State.Paused)
        {
            CurrentState = State.Playing;
            Time.timeScale = 1;
        }
    }

    public void ToMenu()
    {
        CurrentState = State.Menu;
        MobileDirection = Vector2.zero;
        Time.timeScale = 1;
        world.Clear();
        PlayerPrefs.Save();
    }

    private void OnDestroy()
    {
        if (Instance == this) Instance = null;
        Time.timeScale = 1;
    }
}
```

ПП ПМ.13 · Раздел 57

Код · Hazard.cs

1

Назначение и вставка

Входная часть контрольного примера. Обрабатывает взаимодействие игрока с опасностью. Здесь приведён для полноты рабочего примера; повторно добавлять такой компонент не нужно.
Полная версия Hazard.cs приведена далее по частям. Единица использования — весь одноимённый файл; заголовки и номера строк в него не вставлять. В своём проекте читать способ и переносить нужный приём, сохраняя собственную игровую логику.

ПП ПМ.13 · Раздел 58

Hazard.cs · строки 1–16

1

Продолжение полного файла

```
using UnityEngine;

[RequireComponent(typeof(Collider2D))]
public class Hazard : MonoBehaviour
{
    private void Awake()
    {
        GetComponent<Collider2D>().isTrigger = true;
    }

    private void OnTriggerEnter2D(Collider2D other)
    {
        if (!other.TryGetComponent<PlayerController>(out _)) return;
        GameManager.Instance.HitHazard();
    }
}
```

ПП ПМ.13 · Раздел 59

Код · HoldButton.cs

1

Назначение и вставка

Входная часть контрольного примера. Передаёт направление при удержании экранной кнопки и сбрасывает его при отпускании. Здесь приведён для полноты рабочего примера; повторно добавлять такой компонент не нужно.
Полная версия HoldButton.cs приведена далее по частям. Единица использования — весь одноимённый файл; заголовки и номера строк в него не вставлять. В своём проекте читать способ и переносить нужный приём, сохраняя собственную игровую логику.

HoldButton.cs · строки 1–24

1

Продолжение полного файла

```
using UnityEngine;
using UnityEngine.EventSystems;

public class HoldButton : MonoBehaviour,
    IPointerDownHandler, IPointerUpHandler, IPointerExitHandler
{
    public Vector2 direction;
    private bool held;
    public void OnPointerDown(PointerEventData e)
    {
        held = true;
        if (GameManager.Instance.IsPlaying)
            GameManager.Instance.SetDirection(direction);
    }
    public void OnPointerUp(PointerEventData e) => Release();
    public void OnPointerExit(PointerEventData e) => Release();
    private void OnDisable() => Release();
    private void Release()
    {
        if (held && GameManager.Instance != null)
            GameManager.Instance.SetDirection(Vector2.zero);
        held = false;
    }
}
```

Код · LevelManager.cs

1

Назначение и вставка

Входная часть контрольного примера. Хранит параметры уровней и выбирает текущий уровень. Здесь приведён для полноты рабочего примера; повторно добавлять такой компонент не нужно.
Полная версия LevelManager.cs приведена далее по частям. Единица использования — весь одноимённый файл; заголовки и номера строк в него не вставлять. В своём проекте читать способ и переносить нужный приём, сохраняя собственную игровую логику.

LevelManager.cs · строки 1–28

1

Продолжение полного файла

```
using UnityEngine;

[System.Serializable]
public class LevelData
{
    public string title;
    [Range(1, 7)] public int target;
    [Range(1, 6)] public int hazards;
    [Min(20)] public float seconds;
}

public class LevelManager : MonoBehaviour
{
    public LevelData[] levels =
    {
        new() { title = "Первый маршрут", target = 3,
            hazards = 2, seconds = 90 },
        new() { title = "Опасный поворот", target = 5,
            hazards = 4, seconds = 80 },
        new() { title = "Последний портал", target = 7,
            hazards = 6, seconds = 70 }
    };
    public int Current { get; private set; }
    public LevelData Data => levels[Current - 1];

    public bool Select(int number)
    {
        if (number < 1 || number > levels.Length ||
```

ПП ПМ.13 · Раздел 63

LevelManager.cs · строки 29–33

1

Продолжение полного файла

```
            number > SaveService.Unlocked) return false;
        Current = number;
        return true;
    }
}
```

ПП ПМ.13 · Раздел 64

Код · MenuController.cs

1

Назначение и вставка

Изменённый компонент. Создаёт безопасный контейнер, подключает обучение, сведения и захват кадров, добавляет кнопки новых экранов и переходы панелей. Нужен, чтобы новые функции стали доступны через существующее меню.
Полная версия MenuController.cs приведена далее по частям. Единица использования — весь одноимённый файл; заголовки и номера строк в него не вставлять. В своём проекте читать способ и переносить нужный приём, сохраняя собственную игровую логику.

MenuController.cs · строки 1–28

1

Продолжение полного файла

```
using UnityEngine;
using UnityEngine.UI;
using UnityEngine.Events;
using UnityEngine.EventSystems;

public class MenuController : MonoBehaviour
{
    private GameManager game;
    private GameObject menu, hud, pause, result, settings;
    private Text progress, hint, resultTitle, resultText, volumeText;
    [SerializeField] private Button[] levels = new Button[3];
    private Button next;
    private Font font;
    private Transform canvas;
    private TutorialController tutorial;
    private AboutPanel about;

    private void Start()
    {
        game = GameManager.Instance;
        font = Resources.GetBuiltinResource<Font>("LegacyRuntime.ttf");
        var go = new GameObject("Canvas", typeof(Canvas),
            typeof(CanvasScaler), typeof(GraphicRaycaster));
        canvas = go.transform;
        go.GetComponent<Canvas>().renderMode = RenderMode.ScreenSpaceOverlay;
        var scaler = go.GetComponent<CanvasScaler>();
        scaler.uiScaleMode = CanvasScaler.ScaleMode.ScaleWithScreenSize;
        scaler.referenceResolution = new Vector2(1280, 720);
```

MenuController.cs · строки 29–56

1

Продолжение полного файла

```
        scaler.matchWidthOrHeight = 0.5f;
        if (FindAnyObjectByType<EventSystem>() == null)
            new GameObject("EventSystem", typeof(EventSystem),
                typeof(StandaloneInputModule));
        var safe = new GameObject("SafeArea", typeof(RectTransform),
typeof(SafeAreaPanel));
       canvas = safe.GetComponent<SafeAreaPanel>().Initialize((RectTransform)canvas)
;
        tutorial = gameObject.AddComponent<TutorialController>();
        about = gameObject.AddComponent<AboutPanel>();
        gameObject.AddComponent<ScreenshotCapture>();
        BuildMenu();
        BuildHud();
        BuildOverlays();
        about.Initialize(this, canvas);
        tutorial.Initialize(this, canvas);
    }

    private void BuildMenu()
    {
        menu = Panel("MainMenuPanel", canvas, 0, 0, 860, 650);
        Label(menu.transform, "КРИСТАЛЬНЫЙ МАРШРУТ", 0, 260, 800, 50, 34);
        Label(menu.transform, "Три маршрута. Соберите кристаллы и найдите выход.",
            0, 200, 800, 40, 22);
        for (int i = 0; i < 3; i++)
        {
            int number = i + 1;
            levels[i] = Button(menu.transform, "Level" + number,
                "", 0, 115 - i * 85, 640, 66, () => game.StartLevel(number));
        }
```

MenuController.cs · строки 57–84

1

Продолжение полного файла

```
        Button(menu.transform, "Settings", "Настройки звука", 0, -140,
            420, 55, () => ShowSettings(true));
        Button(menu.transform, "Tutorial", "Как играть", -210, -245,
            360, 65, tutorial.Open);
        Button(menu.transform, "About", "Об игре", 210, -245,
            360, 65, about.Open);
    }

    private void BuildHud()
    {
        hud = new GameObject("GamePanel", typeof(RectTransform));
        hud.transform.SetParent(canvas, false);
        var bar = Panel("HUDPanel", hud.transform, 0, 305, 1230, 80);
        progress = Label(bar.transform, "", -70, 0, 1020, 55, 24);
        Button(bar.transform, "PauseButton", "Пауза", 525, 0, 160, 54, game.Pause);
        hint = Label(hud.transform, "", 120, -305, 820, 50, 21);
        Direction("Left", "←", -565, -295, Vector2.left);
        Direction("Down", "↓", -475, -295, Vector2.down);
        Direction("Right", "→", -385, -295, Vector2.right);
        Direction("Up", "↑", -475, -205, Vector2.up);
    }

    private void Direction(string name, string label, float x, float y, Vector2 d)
    {
        var button = Button(hud.transform, name, label, x, y, 80, 80, null);
        button.gameObject.AddComponent<HoldButton>().direction = d;
    }
```

MenuController.cs · строки 85–112

1

Продолжение полного файла

```
private void BuildOverlays()
{
    pause = Panel("PausePanel", canvas, 0, 0, 700, 360);
    Label(pause.transform, "ПАУЗА", 0, 115, 620, 60, 36);
    Button(pause.transform, "Resume", "Продолжить", 0, 15, 420, 65,
        game.TogglePause);
    Button(pause.transform, "Menu", "В меню", 0, -80, 420, 65, game.ToMenu);
    result = Panel("ResultPanel", canvas, 0, 0, 780, 450);
    resultTitle = Label(result.transform, "", 0, 165, 740, 55, 32);
    resultText = Label(result.transform, "", 0, 95, 740, 50, 22);
    next = Button(result.transform, "Next", "Следующий уровень", 0, 10,
        480, 65, () => game.StartLevel(game.Levels.Current + 1));
    Button(result.transform, "Retry", "Повторить уровень", 0, -75,
        480, 65, () => game.StartLevel(game.Levels.Current));
    Button(result.transform, "Menu", "В меню", 0, -160, 480, 65, game.ToMenu);
    settings = Panel("SettingsPanel", canvas, 0, 0, 780, 450);
    Label(settings.transform, "НАСТРОЙКИ", 0, 160, 700, 60, 34);
    volumeText = Label(settings.transform, "", 0, 70, 700, 50, 28);
    Button(settings.transform, "Minus", "−", -140, -20, 120, 65,
        () => SaveService.SetVolume(SaveService.Volume - 0.1f));
    Button(settings.transform, "Plus", "+", 140, -20, 120, 65,
        () => SaveService.SetVolume(SaveService.Volume + 0.1f));
    Button(settings.transform, "Back", "Назад", 0, -140, 420, 65,
        () => ShowSettings(false));
    settings.SetActive(false);
}

private void ShowSettings(bool show)
```

ПП ПМ.13 · Раздел 69

MenuController.cs · строки 113–140

1

Продолжение полного файла

```
    {
        settings.SetActive(show);
        menu.SetActive(!show);
    }

    private void Update()
    {
        if (game == null) return;
        var state = game.CurrentState;
        menu.SetActive(state == GameManager.State.Menu && !settings.activeSelf &&
            !tutorial.IsOpen && !about.IsOpen);
        hud.SetActive(state != GameManager.State.Menu);
        pause.SetActive(state == GameManager.State.Paused);
        bool ended = state == GameManager.State.Won || state ==
GameManager.State.Lost;
        result.SetActive(ended);
        if (state != GameManager.State.Menu) settings.SetActive(false);
        for (int i = 0; i < 3; i++)
        {
            bool unlocked = i + 1 <= SaveService.Unlocked;
            levels[i].interactable = unlocked;
            levels[i].GetComponentInChildren<Text>().text = unlocked
                ? $"Уровень {i + 1}  ·  рекорд {SaveService.Best(i + 1)}"
                : $"Уровень {i + 1}  ·  закрыт";
        }
        volumeText.text = $"Громкость: {Mathf.RoundToInt(SaveService.Volume *
100)}%";
        if (game.Levels.Current == 0) return;
        progress.text = $"Уровень {game.Levels.Current}   Кристаллы " +
            $"{game.Collected}/{game.Levels.Data.target}   Жизни {game.Hearts}" +
```

ПП ПМ.13 · Раздел 70

MenuController.cs · строки 141–168

1

Продолжение полного файла

```
            $"   Время {Mathf.CeilToInt(game.Remaining)}";
        hint.text = game.Hint;
        resultTitle.text = state == GameManager.State.Won
            ? (game.Levels.Current == 3 ? "ИГРА ПРОЙДЕНА" : "УРОВЕНЬ ПРОЙДЕН")
            : "ПОПРОБУЙТЕ ЕЩЁ";
        resultText.text = game.Hint;
        next.gameObject.SetActive(state == GameManager.State.Won &&
game.Levels.Current < 3);
    }

    public GameObject Panel(string name, Transform parent,
        float x, float y, float width, float height)
    {
        var go = new GameObject(name, typeof(RectTransform), typeof(Image));
        Place(go, parent, x, y, width, height);
        go.GetComponent<Image>().color = VisualTheme.Panel;
        if (name.EndsWith("Panel")) go.AddComponent<PanelTransition>();
        return go;
    }

    public Text Label(Transform parent, string value, float x, float y,
        float width, float height, int size)
    {
        var go = new GameObject("Text", typeof(RectTransform), typeof(Text));
        Place(go, parent, x, y, width, height);
        var text = go.GetComponent<Text>();
        text.font = font;
        text.fontSize = size;
        text.color = Color.white;
```

ПП ПМ.13 · Раздел 71

MenuController.cs · строки 169–196

1

Продолжение полного файла

```
        text.alignment = TextAnchor.MiddleCenter;
        text.raycastTarget = false;
        text.text = value;
        return text;
    }

    public Button Button(Transform parent, string name, string label,
        float x, float y, float width, float height, UnityAction action)
    {
        var go = Panel(name, parent, x, y, width, height);
        go.GetComponent<Image>().color = VisualTheme.Accent;
        var button = go.AddComponent<Button>();
        button.targetGraphic = go.GetComponent<Image>();
        if (action != null) button.onClick.AddListener(action);
        Label(go.transform, label, 0, 0, width - 12, height - 8, 24);
        return button;
    }

    private void Place(GameObject go, Transform parent,
        float x, float y, float width, float height)
    {
        go.transform.SetParent(parent, false);
        var rt = go.GetComponent<RectTransform>();
        rt.anchorMin = rt.anchorMax = rt.pivot = new Vector2(0.5f, 0.5f);
        rt.anchoredPosition = new Vector2(x, y);
        rt.sizeDelta = new Vector2(width, height);
    }
}
```

ПП ПМ.13 · Раздел 72

Код · PanelTransition.cs

1

Назначение и вставка

Новый компонент. Управляет прозрачностью панели и доступностью её кнопок при появлении; нужен для заметного перехода между экранами, в том числе на паузе. Подключается методом MenuController.Panel.
Полная версия PanelTransition.cs приведена далее по частям. Единица использования — весь одноимённый файл; заголовки и номера строк в него не вставлять. В своём проекте читать способ и переносить нужный приём, сохраняя собственную игровую логику.

PanelTransition.cs · строки 1–26

1

Продолжение полного файла

```
using UnityEngine;

[RequireComponent(typeof(CanvasGroup))]
public class PanelTransition : MonoBehaviour
{
    public float duration = 0.18f;
    private CanvasGroup group;
    private float elapsed;

    private void OnEnable()
    {
        group = GetComponent<CanvasGroup>();
        elapsed = 0;
        group.alpha = 0;
        group.interactable = false;
        // Block clicks through the appearing panel from the first frame.
        group.blocksRaycasts = true;
    }

    private void Update()
    {
        elapsed += Time.unscaledDeltaTime;
        group.alpha = duration <= 0 ? 1 : Mathf.Clamp01(elapsed / duration);
        group.interactable = group.alpha >= 1;
    }
}
```

ПП ПМ.13 · Раздел 74

Код · PauseController.cs

1

Назначение и вставка

Входная часть контрольного примера. Обрабатывает управление паузой. Здесь приведён для полноты рабочего примера; повторно добавлять такой компонент не нужно.
Полная версия PauseController.cs приведена далее по частям. Единица использования — весь одноимённый файл; заголовки и номера строк в него не вставлять. В своём проекте читать способ и переносить нужный приём, сохраняя собственную игровую логику.

PauseController.cs · строки 1–27

1

Продолжение полного файла

```
using UnityEngine;

public class PauseController : MonoBehaviour
{
    private void Update()
    {
        if (Input.GetKeyDown(KeyCode.Escape))
            GameManager.Instance.TogglePause();
    }

    private void OnApplicationFocus(bool hasFocus)
    {
        if (!hasFocus) GameManager.Instance?.Pause();
    }

    private void OnApplicationPause(bool paused)
    {
        if (paused) GameManager.Instance?.Pause();
        PlayerPrefs.Save();
    }

    // Возврат фокуса не возобновляет игру без действия игрока.
    private void OnDisable()
    {
        Time.timeScale = 1f;
    }
}
```

Код · PlayerController.cs

1

Назначение и вставка

Входная часть контрольного примера. Управляет движением игрока. Здесь приведён для полноты рабочего примера; повторно добавлять такой компонент не нужно.
Полная версия PlayerController.cs приведена далее по частям. Единица использования — весь одноимённый файл; заголовки и номера строк в него не вставлять. В своём проекте читать способ и переносить нужный приём, сохраняя собственную игровую логику.

ПП ПМ.13 · Раздел 77

PlayerController.cs · строки 1–28

1

Продолжение полного файла

```
using UnityEngine;

[RequireComponent(typeof(Rigidbody2D), typeof(Collider2D))]
public class PlayerController : MonoBehaviour
{
    [SerializeField, Min(1f)] private float speed = 5f;
    private Rigidbody2D body;
    private Vector2 startPosition;

    private void Awake()
    {
        body = GetComponent<Rigidbody2D>();
        body.gravityScale = 0f;
        body.freezeRotation = true;
        startPosition = transform.position;
    }

    private void FixedUpdate()
    {
        if (GameManager.Instance == null || !GameManager.Instance.IsPlaying)
        {
            body.linearVelocity = Vector2.zero;
            return;
        }

        Vector2 keyboard = new Vector2(
            ReadAxis(KeyCode.A, KeyCode.LeftArrow, KeyCode.D, KeyCode.RightArrow),
            ReadAxis(KeyCode.S, KeyCode.DownArrow, KeyCode.W, KeyCode.UpArrow));
```

ПП ПМ.13 · Раздел 78

PlayerController.cs · строки 29–49

1

Продолжение полного файла

```
        Vector2 direction = Vector2.ClampMagnitude(
            keyboard + GameManager.Instance.MobileDirection, 1f);
        body.linearVelocity = direction * speed;
    }

    private static float ReadAxis(
        KeyCode negativeA, KeyCode negativeB,
        KeyCode positiveA, KeyCode positiveB)
    {
        float value = 0f;
        if (Input.GetKey(negativeA) || Input.GetKey(negativeB)) value -= 1f;
        if (Input.GetKey(positiveA) || Input.GetKey(positiveB)) value += 1f;
        return value;
    }

    public void ReturnToStart()
    {
        body.linearVelocity = Vector2.zero;
        body.position = startPosition;
    }
}
```

ПП ПМ.13 · Раздел 79

Код · SafeAreaPanel.cs

1

Назначение и вставка

Новый компонент. Создаёт SafeContent и вписывает интерфейс в безопасную область экрана, чтобы края и вырезы не закрывали элементы управления. Создаётся и настраивается из MenuController.Start.
Полная версия SafeAreaPanel.cs приведена далее по частям. Единица использования — весь одноимённый файл; заголовки и номера строк в него не вставлять. В своём проекте читать способ и переносить нужный приём, сохраняя собственную игровую логику.

SafeAreaPanel.cs · строки 1–28

1

Продолжение полного файла

```
using UnityEngine;

// Keep a 1280x720 UI layout inside the device safe area.
public class SafeAreaPanel : MonoBehaviour
{
    public bool simulateInsets;
    private RectTransform area, content;

    public RectTransform Initialize(RectTransform parent)
    {
        area = (RectTransform)transform;
        area.SetParent(parent, false);
        content = new GameObject("SafeContent", typeof(RectTransform))
            .GetComponent<RectTransform>();
        content.SetParent(area, false);
        content.anchorMin = content.anchorMax = new Vector2(0.5f, 0.5f);
        content.sizeDelta = new Vector2(1280, 720);
        Apply();
        return content;
    }

    public void Apply()
    {
        if (content == null || Screen.width == 0 || Screen.height == 0) return;
        Rect safe = Screen.safeArea;
        if (simulateInsets)
            safe = new Rect(Screen.width * 0.08f, Screen.height * 0.05f,
                Screen.width * 0.84f, Screen.height * 0.9f);
```

ПП ПМ.13 · Раздел 81

SafeAreaPanel.cs · строки 29–46

1

Продолжение полного файла

```
        ApplyRect(safe, new Vector2(Screen.width, Screen.height));
    }

    public void ApplyRect(Rect safe, Vector2 screen)
    {
        if (content == null || screen.x <= 0 || screen.y <= 0) return;
        area.anchorMin = new Vector2(safe.xMin / screen.x, safe.yMin / screen.y);
        area.anchorMax = new Vector2(safe.xMax / screen.x, safe.yMax / screen.y);
        area.offsetMin = area.offsetMax = Vector2.zero;
        var parent = (RectTransform)area.parent;
        float width = parent.rect.width * safe.width / screen.x;
        float height = parent.rect.height * safe.height / screen.y;
        float scale = Mathf.Min(width / 1280f, height / 720f);
        content.localScale = Vector3.one * scale;
    }

    private void LateUpdate() => Apply();
}
```

ПП ПМ.13 · Раздел 82

Код · SaveService.cs

1

Назначение и вставка

Входная часть контрольного примера. Хранит и читает локальные настройки и игровой прогресс. Здесь приведён для полноты рабочего примера; повторно добавлять такой компонент не нужно.
Полная версия SaveService.cs приведена далее по частям. Единица использования — весь одноимённый файл; заголовки и номера строк в него не вставлять. В своём проекте читать способ и переносить нужный приём, сохраняя собственную игровую логику.

ПП ПМ.13 · Раздел 83

SaveService.cs · строки 1–28

1

Продолжение полного файла

```
using UnityEngine;

public static class SaveService
{
    // Свой префикс не затрагивает сохранения других учебных проектов.
    private const string Prefix = "PP12.Crystal.v1.";
    public static int Unlocked => Mathf.Clamp(
        PlayerPrefs.GetInt(Prefix + "Unlocked", 1), 1, 3);
    public static float Volume => Mathf.Clamp01(
        PlayerPrefs.GetFloat(Prefix + "Volume", 0.7f));

    public static int Best(int level) => Mathf.Max(0,
        PlayerPrefs.GetInt(Prefix + "Best" + level, 0));

    public static void Complete(int level, int score)
    {
        PlayerPrefs.SetInt(Prefix + "Unlocked",
            Mathf.Max(Unlocked, Mathf.Min(level + 1, 3)));
        if (score > Best(level))
            PlayerPrefs.SetInt(Prefix + "Best" + level, score);
        PlayerPrefs.Save();
    }

    public static void SetVolume(float value)
    {
        value = Mathf.Clamp01(value);
        AudioListener.volume = value;
        PlayerPrefs.SetFloat(Prefix + "Volume", value);
```

ПП ПМ.13 · Раздел 84

SaveService.cs · строки 29–31

1

Продолжение полного файла

```
        PlayerPrefs.Save();
    }
}
```

ПП ПМ.13 · Раздел 85

Код · ScreenshotCapture.cs

1

Назначение и вставка

Новый компонент. По F9 сохраняет текущий кадр в PNG для отчёта и материалов представления игры. В игре подключается из MenuController.Start; для рекламной графики используется редакторский ReleaseAssets.
Полная версия ScreenshotCapture.cs приведена далее по частям. Единица использования — весь одноимённый файл; заголовки и номера строк в него не вставлять. В своём проекте читать способ и переносить нужный приём, сохраняя собственную игровую логику.

ПП ПМ.13 · Раздел 86

ScreenshotCapture.cs · строки 1–28

1

Продолжение полного файла

```
using System;
using System.Collections;
using System.IO;
using UnityEngine;

public class ScreenshotCapture : MonoBehaviour
{
    public string LastPath { get; private set; }
    private bool busy;

    private void Update()
    {
        if (Input.GetKeyDown(KeyCode.F9)) Capture();
    }

    public void Capture()
    {
        if (!busy) StartCoroutine(SaveFrame());
    }

    private IEnumerator SaveFrame()
    {
        busy = true;
        yield return new WaitForEndOfFrame();
        string folder = Path.Combine(Application.persistentDataPath, "Screenshots");
        Directory.CreateDirectory(folder);
        LastPath = Path.Combine(folder,
            "crystal-" + DateTime.Now.ToString("yyyyMMdd-HHmmss-fff") + ".png");
```

ПП ПМ.13 · Раздел 87

ScreenshotCapture.cs · строки 29–33

1

Продолжение полного файла

```
        ScreenCapture.CaptureScreenshot(LastPath);
        Debug.Log("Screenshot requested: " + LastPath);
        busy = false;
    }
}
```

ПП ПМ.13 · Раздел 88

Код · TutorialController.cs

1

Назначение и вставка

Новый компонент. Создаёт три вводные карточки, переходы и пропуск, сохраняет факт просмотра. Нужен для знакомства с правилами и их повторного чтения через меню. Подключается из MenuController.Start. Полная версия TutorialController.cs приведена далее по частям. Единица использования — весь одноимённый файл; заголовки и номера строк в него не вставлять. В своём проекте читать способ и переносить нужный приём, сохраняя собственную игровую логику.

TutorialController.cs · строки 1–28

1

Продолжение полного файла

```
using UnityEngine;
using UnityEngine.UI;

public class TutorialController : MonoBehaviour
{
    public const string SeenKey = "PP13.Crystal.Tutorial.v1";
    private readonly string[] steps =
    {
        "1 / 3 · ДВИЖЕНИЕ\nУдерживайте экранную стрелку. Отпустите её, чтобы
остановиться.",
        "2 / 3 · ЦЕЛЬ\nСоберите все жёлтые ромбы. Счётчик вверху показывает
собранное и цель.",
        "3 / 3 · ОПАСНОСТЬ И ВЫХОД\nКрасный крест отнимает жизнь. Соберите цель и
войдите в зелёный портал."
    };
    private GameObject panel;
    private Text message, nextLabel;
    private int index;
    public bool IsOpen => panel != null && panel.activeSelf;
    public int Step => index;

    public void Initialize(MenuController ui, Transform parent)
    {
        panel = ui.Panel("TutorialPanel", parent, 0, 0, 1000, 580);
        ui.Label(panel.transform, "КАК ИГРАТЬ", 0, 220, 900, 60, 36);
        message = ui.Label(panel.transform, "", 0, 45, 900, 230, 30);
        var next = ui.Button(panel.transform, "TutorialNext", "Далее", 200, -200,
            360, 70, Advance);
        nextLabel = next.GetComponentInChildren<Text>();
        ui.Button(panel.transform, "TutorialSkip", "Пропустить", -200, -200,
            360, 70, Close);
```

TutorialController.cs · строки 29–56

1

Продолжение полного файла

```
        panel.SetActive(false);
        if (PlayerPrefs.GetInt(SeenKey, 0) == 0) Open();
    }

    public void Open()
    {
        if (GameManager.Instance.CurrentState != GameManager.State.Menu) return;
        index = 0;
        panel.SetActive(true);
        panel.transform.SetAsLastSibling();
        Refresh();
    }

    public void Advance()
    {
        if (!IsOpen) return;
        if (index < steps.Length - 1) { index++; Refresh(); }
        else Close();
    }

    public void Close()
    {
        PlayerPrefs.SetInt(SeenKey, 1);
        PlayerPrefs.Save();
        panel.SetActive(false);
    }

    private void Refresh()
```

ПП ПМ.13 · Раздел 91

TutorialController.cs · строки 57–61

1

Продолжение полного файла

```
    {
        message.text = steps[index];
        nextLabel.text = index == steps.Length - 1 ? "В меню" : "Далее";
    }
}
```

ПП ПМ.13 · Раздел 92

Код · VisualTheme.cs

1

Назначение и вставка

Входная часть контрольного примера. Хранит общие цвета оформления. Здесь приведён для полноты рабочего примера; повторно добавлять такой компонент не нужно.
Полная версия VisualTheme.cs приведена далее по частям. Единица использования — весь одноимённый файл; заголовки и номера строк в него не вставлять. В своём проекте читать способ и переносить нужный приём, сохраняя собственную игровую логику.

ПП ПМ.13 · Раздел 93

VisualTheme.cs · строки 1–16

1

Продолжение полного файла

```
using UnityEngine;

public static class VisualTheme
{
    public static readonly Color Background = new(0.035f, 0.055f, 0.10f);
    public static readonly Color Surface = new(0.07f, 0.10f, 0.18f);
    public static readonly Color Panel = new(0.055f, 0.075f, 0.13f, 0.96f);
    public static readonly Color Button = new(0.12f, 0.18f, 0.30f, 0.94f);
    public static readonly Color Accent = new(0.16f, 0.55f, 0.75f, 0.98f);
    public static readonly Color Player = new(0.19f, 0.88f, 1f);
    public static readonly Color Collectible = new(1f, 0.78f, 0.16f);
    public static readonly Color Danger = new(1f, 0.25f, 0.28f);
    public static readonly Color Success = new(0.25f, 0.82f, 0.45f);
    public static readonly Color PrimaryText = Color.white;
    public static readonly Color SecondaryText = new(0.82f, 0.88f, 0.98f);
}
```

ПП ПМ.13 · Раздел 94

Код · WorldBuilder.cs

1

Назначение и вставка

Изменённый компонент. Сохраняет создание поля и объектов; добавляет подстройку обзора камеры по пропорциям окна, чтобы игровое поле не обрезалось. В своём варианте сохраните собственную расстановку и правила.

Полная версия WorldBuilder.cs приведена далее по частям. Единица использования — весь одноимённый файл; заголовки и номера строк в него не вставлять. В своём проекте читать способ и переносить нужный приём, сохраняя собственную игровую логику.

WorldBuilder.cs · строки 1–28

1

Продолжение полного файла

```
using System.Collections.Generic;
using UnityEngine;

public class WorldBuilder : MonoBehaviour
{
    public PlayerController Player { get; private set; }
    private GameObject root;
    private readonly Dictionary<int, Sprite> sprites = new();
    private readonly Vector2[] crystals =
    {
        new(-5.7f, 2.8f), new(-2.5f, 0.8f), new(0.8f, 3),
        new(3.2f, -1.8f), new(5.7f, 2.2f), new(-1, -3), new(6, -3)
    };
    private readonly Vector2[] hazards =
    {
        new(-4, -1.8f), new(-0.6f, -0.2f), new(2.2f, 1.5f),
        new(5, -0.5f), new(-6, 0), new(0, -3.5f)
    };

    public void BuildCamera()
    {
        Camera cam = Camera.main;
        if (cam == null)
        {
            var go = new GameObject("Main Camera");
            go.tag = "MainCamera";
            cam = go.AddComponent<Camera>();
        }
```

ПП ПМ.13 · Раздел 96

WorldBuilder.cs · строки 29–56

1

Продолжение полного файла

```
        cam.orthographic = true;
        cam.orthographicSize = Mathf.Max(6.2f, 9f / cam.aspect);
        cam.transform.position = new Vector3(0, 0, -10);
        cam.clearFlags = CameraClearFlags.SolidColor;
        cam.backgroundColor = VisualTheme.Background;
        if (!cam.GetComponent<AudioListener>())
            cam.gameObject.AddComponent<AudioListener>();
    }

    private void LateUpdate()
    {
        if (Camera.main != null)
            Camera.main.orthographicSize = Mathf.Max(6.2f, 9f / Camera.main.aspect);
    }

    public void Clear()
    {
        if (!root) return;
        root.SetActive(false);
        Destroy(root);
    }

    public void Build(LevelData data)
    {
        Clear();
        root = new GameObject("LevelObjects");
        Item("Field", Vector2.zero, new(17, 9.5f), VisualTheme.Surface, 0, -10);
        Wall("Top", new(0, 4.7f), new(17, 0.3f));
```

ПП ПМ.13 · Раздел 97

WorldBuilder.cs · строки 57–84

1

Продолжение полного файла

```
        Wall("Bottom", new(0, -4.7f), new(17, 0.3f));
        Wall("Left", new(-8.4f, 0), new(0.3f, 9.6f));
        Wall("Right", new(8.4f, 0), new(0.3f, 9.6f));
        Wall("A", new(-3.8f, 1.2f), new(0.35f, 3));
        if (data.target >= 5)
            Wall("B", new(1.4f, -2.2f), new(3.2f, 0.35f));
        if (data.target >= 7)
            Wall("C", new(4.4f, 2), new(0.35f, 2.5f));
        var player = Item("Player", new(-6.5f, -3.1f), new(0.7f, 0.7f),
            VisualTheme.Player, 1, 10);
        player.AddComponent<Rigidbody2D>();
        player.AddComponent<CircleCollider2D>();
        Player = player.AddComponent<PlayerController>();
        for (int i = 0; i < Mathf.Clamp(data.target, 1, crystals.Length); i++)
        {
            var c = Item("Crystal" + (i + 1), crystals[i], new(0.6f, 0.8f),
                VisualTheme.Collectible, 2, 5);
            c.AddComponent<PolygonCollider2D>();
            c.AddComponent<CollectibleVisual>();
            c.AddComponent<Collectible>();
        }
        for (int i = 0; i < Mathf.Clamp(data.hazards, 1, hazards.Length); i++)
        {
            var h = Item("Hazard" + (i + 1), hazards[i], new(0.8f, 0.8f),
                VisualTheme.Danger, 3, 4);
            h.AddComponent<CircleCollider2D>();
            h.AddComponent<Hazard>();
        }
```

WorldBuilder.cs · строки 85–112

1

Продолжение полного файла

```
        var exit = Item("Exit", new(6.5f, 3.1f), new(1, 1),
            VisualTheme.Success, 4, 2);
        exit.AddComponent<BoxCollider2D>();
        exit.AddComponent<ExitZone>();
    }

    private void Wall(string name, Vector2 p, Vector2 scale)
    {
        Item("Wall" + name, p, scale, VisualTheme.Button, 0, 1)
            .AddComponent<BoxCollider2D>();
    }

    private GameObject Item(string name, Vector2 p, Vector2 scale,
        Color color, int shape, int order)
    {
        var go = new GameObject(name);
        go.transform.SetParent(root.transform);
        go.transform.position = p;
        go.transform.localScale = scale;
        var sr = go.AddComponent<SpriteRenderer>();
        sr.sprite = GetSprite(shape);
        sr.color = color;
        sr.sortingOrder = order;
        return go;
    }

    private Sprite GetSprite(int shape)
    {
```

WorldBuilder.cs · строки 113–140

1

Продолжение полного файла

```
        if (sprites.TryGetValue(shape, out Sprite cached)) return cached;
        const int size = 64;
        var texture = new Texture2D(size, size);
        for (int y = 0; y < size; y++)
        for (int x = 0; x < size; x++)
        {
            float a = (x + 0.5f) / size * 2 - 1;
            float b = (y + 0.5f) / size * 2 - 1;
            bool inside = shape switch
            {
                1 => a * a + b * b < 0.9f,
                2 => Mathf.Abs(a) + Mathf.Abs(b) < 0.92f,
                3 => Mathf.Abs(a - b) < 0.2f || Mathf.Abs(a + b) < 0.2f,
                4 => Mathf.Max(Mathf.Abs(a), Mathf.Abs(b)) > 0.65f,
                _ => true
            };
            texture.SetPixel(x, y, inside ? Color.white : Color.clear);
        }
        texture.Apply();
        var sprite = Sprite.Create(texture, new Rect(0, 0, size, size),
            new Vector2(0.5f, 0.5f), size);
        sprites[shape] = sprite;
        return sprite;
    }

    private void OnDestroy()
    {
        foreach (Sprite sprite in sprites.Values)
```

ПП ПМ.13 · Раздел 100

WorldBuilder.cs · строки 141–146

1

Продолжение полного файла

```
        {
            Destroy(sprite.texture);
            Destroy(sprite);
        }
    }
}
```